Project – Greatest Common Divisor

The proposed project for this course is an N-bit module which computes the Greatest Common Divisor (GCD). This will serve as a new functional unit in our enhanced ALU.

Your assignment involves carrying out a comprehensive verification process for the specified design. The following section contains the architectural specifications of the module. Using this information, along with the provided code, you are expected to execute the various verification phases and methodologies discussed in the course.

1. Get Ready for this Project

In the web of the course at https://docencia.ac.upc.edu/master/MIRI/PV/project.html, you will find a package gcd.tgz containing the following file/s:

a) gcd.sv: The design in SystemVerilog language for the DUT.

2. Specification

2.1. Overview

The gcd module computes the Greatest Common Divisor of two unsigned integers using the subtractive Euclidean algorithm. It processes one input pair at a time and communicates through two independent ready/valid handshake channels.

Mathematical Definition

For unsigned operands A and B:

```
gcd(A, 0) = A            // covers gcd(0, 0) = 0
gcd(0, B) = B
gcd(A, A) = A
gcd(A, B) = gcd(A – B, B)    when A > B
gcd(A, B) = gcd(A, B – A)    when B > A
```

Operation Sequence

- Module waits in IDLE state, asserting in_ready = 1.
- On the input handshake, operands are latched. The result is determined immediately for zero or equal operands; otherwise, iterative subtraction begins.
- When convergence is reached, the result is latched into the output register and the module enters DONE, asserting out_valid = 1.
- After the output handshake completes, the module returns to IDLE.

2.2. Parameters

Parameter	Description	Default
`WIDTH`	Width of the data operands	16 (must be ≥ 1)

2.3. Top-Level Interface

Clock and Reset

Signal	Direction	Width	Description
`clk`	Input	1	Clock signal
`rst_n`	Input	1	Active-low sync reset

Input Channel

Signal	Direction	Width	Description
`in_valid`	Input	1	Asserted by the producer when a_in and b_in hold valid data
`in_ready`	Output	1	Asserted by the module when it can accept a new transaction. High only in the IDLE state.
`a_in`	Input	WIDTH	First operand
`b_in`	Input	WIDTH	Second operand

Handshake: A transaction is captured on the rising edge where in_valid = 1 and in_ready = 1 simultaneously. The producer must hold a_in and b_in stable throughout any period where in_valid = 1 and in_ready = 0.

Output Channel

Signal	Direction	Width	Description
`out_valid`	Output	1	Asserted by the module when gcd_out holds a valid result. High for the entire DONE state.
`out_ready`	Input	1	Asserted by the consumer when it is ready to accept the result.
`gcd_out`	Output	WIDTH	GCD result. Valid and stable for the entire period that out_valid is asserted.

Handshake: The transaction completes on the rising edge where out_valid = 1 and out_ready = 1 simultaneously. gcd_out must be stable and correct from the first cycle out_valid is asserted until the handshake completes.

gcd_out must be driven from a dedicated result register that is latched on the transition edge into DONE — not from the working registers a_reg / b_reg.

2.4. Functional Description

The behavior is driven based on an FSM with three states:

State	in_ready	out_valid	Description
`IDLE`	1	0	Waiting for input. Ready to accept a new transaction.
`RUN`	0	0	Iterative subtraction in progress. No new input accepted.
`DONE`	0	1	Result ready. Waiting for the consumer to accept it.

Each cycle in RUN:

- if a_reg > b_reg then a_next = a_reg − b_reg and b_next = b_reg
- else if b_reg > a_reg then b_next = b_reg − a_reg and a_next = a_reg

Transitions

- IDLE → DONE: Input handshake AND (a_in == 0 OR b_in == 0 OR a_in == b_in). Result is known immediately. This transition takes exactly 1 cycle.
- IDLE → RUN: Input handshake AND (a_in ≠ 0 AND b_in ≠ 0 AND a_in ≠ b_in).
- RUN → DONE: The next-cycle values of the working registers are equal (a_next == b_next). This transition fires on the same clock edge that performs the final subtraction step — the termination check is evaluated against the combinatorial next-cycle register values, not the registered values from the previous cycle.
- DONE → IDLE: Output handshake (out_valid = 1 AND out_ready = 1).

2.5. Behavior

Reset

- Active-low, synchronous.
- rst_n is sampled on the rising edge of clk.
- It must not appear in the always_ff sensitivity list.

While rst_n = 0, on every rising clock edge:

- state ← IDLE
- All internal registers ← 0
- in_ready = 1, out_valid = 0, gcd_out = 0

On the first rising edge after rst_n is released, the module is in IDLE and ready to accept input immediately.

Latency

Latency is defined as the number of cycles from the input handshake edge to the first cycle where out_valid is 1.

Input condition	Latency	Notes
Early exit: a_in == 0 OR b_in == 0 OR a_in == b_in	1 cyc	Special case: IDLE → DONE directly
General case	1 + N cyc	N >=1 subtraction steps

Throughput

One transaction at a time. A new input handshake may occur on the earliest cycle after the output handshake returns the module to IDLE.

Illegal / Undefined conditions

Changing a_in or b_in while in_valid = 1 and in_ready = 0, or de-asserting in_valid before the handshake completes, are protocol violations. Behavior is undefined.

3. What is Expected

The following is expected to be found in the testbench/verification report:

- A detailed verification testplan: including a description of the design, a verification traceability matrix (requirement-feature-test), the verification strategy selected, a diagram of your testbench architecture, the coverage goals, and the closure criteria.
- A complete UVM testbench.
- At least three uvm_tests based on respective uvm_sequences for:
    - Bring-up tests, random tests, and directed tests.
    - Command line examples to run each.
- A code/functional coverage report (spawn over the time: after applying each uvm_test as defined before)
- One verification failure found: force a bug in the design (any) and check that your test/s fail/s
    - Write an imaginary bug report ticket on how you would report the issue found to the design team
    - You can include waveforms whenever they help clarify or describe it.

4. What to Turn In

Upon conclusion of your project, please submit the following in FIB El Racó:

1. Your complete testbench source code (in a single file)
2. A PDF document with the following:
    a. Please indicate how many hours you dedicated to this lab. This will not affect your grade but will be helpful for calibrating the workload for the future.
    b. The verification report.

Criteria for evaluation include:

- A comprehensive verification plan that aligns perfectly with the testbench
- Clarity and a well-organized testbench
- Completeness and correctness
- AI being used collaboratively, not to fully write the testplan, testbench (or parts)